zgz-pileup — usage guide

`zgz-pileup` finds the single pulses and resolves the piled-up pulses in the triggered records of the ZGZ acquisition (`c<ch>.txt` + `out.txt`), and writes the **time and energy of every pulse**, clean or recovered from pile-up, to one table per run and channel. Pile-up is resolved with template matching (TM): every record is fitted as a sum of pulses whose shape, including the long tail and the undershoot below baseline, is measured from the data itself.

What the method does and why is explained in the accompanying report (*Template-matching recovery of pile-up in the ZGZ RUN1 records*). How the code does it, step by step, with every fixed setting and where it is, is in `docs/code_guide.md` (also as PDF). This guide is only about running it.

---

1. Install

You need Python 3.9 or newer on Linux (macOS should work too; Windows is untested). The only dependencies are numpy, scipy, pandas and matplotlib.

With conda (recommended):

```
cd zgz-pileup
conda env create -f environment.yml      # creates the environment "zgz-pileup"
conda activate zgz-pileup
```

With pip only:

```
cd zgz-pileup
python3 -m venv .venv && source .venv/bin/activate
pip install numpy scipy pandas matplotlib
```

There is nothing else to install: the scripts find the `zgzpileup` folder next to them.

2. Input data and settings

Put the files of each run in one folder:

```
RUN1/
  c0.zip   c1.zip   c2.zip     (or c0.txt c1.txt c2.txt: either works, the zips are read
directly)
  out.txt
```

- `c<ch>.txt`: one triggered record per block, blocks separated by an empty line, three columns per sample: time since the start of the record (s), height, VST filter output. The sampling period and the height-to-ADC step are detected automatically.
- `out.txt`: one row per record of every channel: channel, time since the start of the run (s), energy (VST filter maximum). The records are matched to these rows by energy, which gives every record its time.

Then edit **`config.py`**, the only file with settings:

```
RUNS = {                      # a name of your choice -> the folder of that run
    "RUN1": "/data/ZGZ/RUN1",
    "RUN2": "/data/ZGZ/RUN2",
}
CHANNELS = [0, 1, 2]
WORK = "/data/ZGZ/zgz-work"   # where all outputs go
POOL_RUNS = True              # learn one model per channel from the clean pulses of all
runs
PEAK_KEV = 766                # energy of the main peak, for the keV calibration (None:
amplitudes only)
WORKERS = 3                   # processes for the TM fits
MAX_TEMP = 85                 # the fits pause while the CPU is hotter than this (°C)
```

Several runs. The pulse shape, tail and templates belong to a detector channel, not to a run, so with `POOL_RUNS = True` the clean pulses of all runs are pooled per channel to learn them (one model per channel), and every run is then fitted with the model of its channel. This is right as long as the gain did not change between runs: `02_clean_pulses.py` prints the photopeak position of every run and warns if it moves by more than 1 %. In that case set `POOL_RUNS = False`, and every run learns its own model.

You need about 2 GB of free disk space per run and channel in `WORK`, and about 6 GB of memory. A run of three channels like RUN1 takes about 45 minutes on a desktop computer.

3. Run

The analysis is eight scripts, run in order. Each one reads what the previous ones wrote, prints what it found and draws its figures, so you can stop after any step and look:

Script	What it does	Writes	Time (RUN1, one channel)
`01_read_data.py`	reads c<ch> into a compact file and matches every record to its `out.txt` row (time)	`records.npz`, `times.npz`	~4 min
`02_clean_pulses.py`	sorts every record into a class (clean, pile-up, ...); pulse shape and amplitudes from the clean pulses	`pulses.npz`, `shape.npz`	~2 min
`03_tail.py`	measures the pulse tail and undershoot from the baseline of the next record	`tail.npz`	< 1 min
`04_tm_params.py`	chooses the TM trigger: fast-filter length and threshold	`tm_params.json`	< 1 min
`05_train_tm.py`	trains the TM templates	`tm_model.npz`	< 1 min
`06_calibrate.py`	amplitude → keV, from `PEAK_KEV`	`calibration.json`	< 1 min
`07_benchmark.py`	TM on synthetic pile-up made of real pulses, with known truth	`benchmark.csv`	< 1 min
`08_recover.py`	TM on every record that is not clean; writes the result table	`events.csv`	~7 min with 3 workers

```
python scripts/01_read_data.py
python scripts/02_clean_pulses.py
...
python scripts/08_recover.py
```

or all of them at once:

```
./run_all.sh
```

Every script takes `--channels` and `--runs` to work on part of the data, for example `python scripts/08_recover.py --runs RUN2 --channels 1`. A script only needs the outputs of the scripts before it, so after a crash or a change you rerun from that script on. To change only the calibration, rerun `06_calibrate.py` and `08_recover.py`. To try the last step quickly, `python scripts/08_recover.py --limit 2000` fits only 2000 records.

Where the outputs go. Inside `WORK`:

```
runs/<run>/ch<c>/      one run of one channel: records.npz, times.npz, pulses.npz,
events.csv
models/all/ch<c>/      the model of a channel: shape.npz, tail.npz, tm_model.npz,
calibration.json, benchmark.csv
                       (models/<run>/ch<c>/ with POOL_RUNS = False)
```

Each folder has a `figures/` subfolder and a `summary.json` with the numbers of its steps.

The energy calibration is proportional, E = c·A: `c` puts the photopeak of the clean pulses at `PEAK_KEV`. For RUN1 the main peak is at about 766 keV on the existing energy scale of the acquisition; if you know the line, give its true energy. As a check, `06_calibrate.py` also finds the sum peak (two photopeak pulses detected together), which should come out at about twice that energy.

4. The result: `events.csv`

One file per run and channel, `WORK/runs/<run>/ch<c>/events.csv`, one row per pulse. Columns:

Column	Meaning
`record`	record number in `c<ch>.txt` (0 = first block)
`out_row`	matching row of this channel in `out.txt` (0-based; −1 if the record has no row)
`kind`	`clean` (selected single pulse; amplitude from the mean shape), `fitted` (pulse found by TM in a record that is not clean), `baseline` (record that starts on the tail of an earlier pulse, fitted by TM), `saturated` (pulse above the ADC range: time only)
`n_pulses`	pulses in this record: `fitted` rows with `n_pulses` ≥ 2 are pile-up resolved by TM
`pulse`	0, 1, 2 … in time order within the record
`time_s`	pulse time since the start of the run (s): the record time from `out.txt` plus the pulse position in the record
`time_in_record_us`	pulse position in the record (µs), at its 50 % rise
`amplitude_adc`	pulse amplitude (ADC counts)
`energy_kev`	`amplitude_adc` converted with `calibration.json` (empty without a calibration)
`chi2ndf`	χ^2/ndf of the TM fit of the record (empty for clean pulses, which are not fitted by TM)
`relative_area`	clean pulses only: area divided by the single-pulse curve of area vs amplitude; about 1 for the main pulse shape, clearly lower (0.6–0.85, depending on the channel) for the second, faster-decaying shape
`flag`	empty for normal pulses; `outside_record` (fitted position outside the record: ignore), `amplitude_at_bound` (amplitude at the fit limit: unreliable), `saturated`

For spectra and time distributions use the rows with an empty `flag`. Example with pandas:

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt

ev = pd.read_csv("zgz-work/runs/RUN1/ch1/events.csv")
ok = ev[ev.flag.isna()]

# energy spectra of clean pulses and of pulses resolved in pile-up records
bins = np.linspace(0, 2000, 400)
pileup = ok[(ok.kind == "fitted") & (ok.n_pulses >= 2)]
plt.hist(ok[ok.kind == "clean"].energy_kev, bins, histtype="step", label="clean")
plt.hist(pileup.energy_kev, bins, histtype="step", label="resolved pile-up")
plt.yscale("log"); plt.xlabel("keV"); plt.legend(); plt.show()

# time to the previous pulse, all pulses of the channel
t = np.sort(ok.time_s.dropna().to_numpy())
plt.hist(np.diff(t) * 1e6, np.logspace(-1, 8, 180)); plt.xscale("log"); plt.yscale("log")
plt.xlabel("µs since the previous pulse"); plt.show()
```

To combine channels, read each `events.csv`, add a `channel` column and concatenate: `time_s` is on the common run clock of `out.txt`. Times of different runs are not on a common clock.

Two pulse shapes. The data contain a second population of single pulses that decay faster than the main ones (a few % of the clean pulses in channel 1, a quarter to a third in channels 0 and 2). The package keeps both together, so the templates are a mixture of the two shapes and the energies of second-shape pulses are approximate (see the report). For clean pulses, `relative_area < 0.92` selects the second shape.

5. Check the results

`python scripts/checks.py` runs three checks on your own data, without using any of our numbers: the pairing of records with `out.txt` rows, the fitter on records built from known pulses, and the photopeak of clean against resolved pulses. `docs/code_guide.md`, section 10, says what to expect.

Then look at the figures (in the `figures/` folder next to each output):

Figure	Folder	What to check
`01_record_gaps.png`	run	the time between records looks like a counting experiment (bursts are normal)
`02_clean_selection.png`	run	panel (a): single pulses form one curve of area against amplitude, inside the band
`03_record_classes.png`	run	the clean examples are single pulses; the pile-up classes really contain pile-up
`04_clean_pulses.png`	model	panel (b): one narrow band of pulse shapes
`05_tail_undershoot.png`	model	the measured tail joins the clean-pulse shape smoothly; the template (panel b) crosses zero and returns towards it
`06_tm_settings.png`	model	panel (b): almost all clean pulses give exactly one trigger at the chosen threshold
`07_energy_calibration.png`	model	the photopeak is at `PEAK_KEV` and the sum peak at about twice that energy
`08_benchmark.png`	model	detection and energy and time errors against the delay between two pulses
`09_pileup_fits_random.png`, `10_pileup_fits_crowded.png`	run	the TM fits follow the data
`11_energy_spectra.png`, `12_time_distributions.png`	run	the results
`13_checks_fitter.png`	model	made by `checks.py`: the fits of records with known pulses

6. Run time, CPU and memory

The long step is `08_recover.py`, which fits every record that is not clean (about 10 % of the records: pile-up, records outside the area band, and records that start on an earlier pulse's tail). Its time scales with 1/`WORKERS`. Each worker is one CPU core at full load. The fits pause by themselves while the CPU is hotter than `MAX_TEMP` (only on machines where the temperature can be read). If the machine is shared, keep `WORKERS` well below the number of cores.

Memory: about 6 GB in `02_clean_pulses.py` and `05_train_tm.py` for a 25 GB `c<ch>.txt`.

7. Settings

Besides `config.py`, all settings are measured from the data of each channel (fast-filter length, trigger threshold, template window, residual threshold, tail). They are printed during the run and stored in `summary.json` and `tm_settings.json`. The fixed rules (what counts

as a clean pulse, the hold-out fraction, …) are constants at the top of the scripts and of `zgzpileup/pulses.py`, `zgzpileup/tail.py` and `zgzpileup/tm.py`, each with a comment; changing them is not needed for this data.

8. Troubleshooting

- **`no c1.zip or c1.txt in …`**: check the folder of that run in `config.py` and the file names.
- **`… not found: run the script that makes it first`**: a script needs the output of an earlier one; run that one first.
- **Few records matched** in `01_read_data.py` (it prints the fraction): `out.txt` is from a different run, or the channel number is wrong.
- **`WARNING: the photopeak moves by … between runs`**: the gain changed; set `POOL_RUNS = False`.
- **`only N record pairs for the tail measurement`**: too few records start shortly after another one to measure the tail; this needs a run with some pile-up.
- **Out of memory**: close other programs, or run one channel at a time (`--channels 1`).

9. Optional: export the clean pulses

For other tools that need every pulse in full (for example to train templates or a neural network on synthetic pile-up), `scripts/export_clean_pulses.py` writes clean pulses as fixed-length records, each one completed after the end of its record with the measured tail and undershoot:

```
python scripts/export_clean_pulses.py --runs RUN1 --channels 1 --n 100000
```

It writes `runs/<run>/ch<c>/export/clean_pulses_<N>.dat` (float32, no header, N samples per pulse), `clean_pulses_info.npz` (record number and amplitude of each pulse) and a `README.txt` with the format. 100,000 pulses take 0.6–1.2 GB, depending on the length of the longest record of the channel. This is not needed for `events.csv`.

10. Files of the package

```
config.py              the settings (runs, channels, output folder, peak energy, workers)
run_all.sh             runs the eight scripts in order
scripts/               the steps, 01_read_data.py … 08_recover.py, checks.py and
export_clean_pulses.py
docs/code_guide.md     how the code works, step by step
zgzpileup/
  io.py                reading c<ch>.txt / zip and out.txt; record ↔ out.txt matching
  pulses.py            selection of the clean pulses, pulse shape, amplitudes
  tail.py              tail and undershoot from the next-record baselines
  tm.py                TM settings, templates, fitting (in parallel)
  fitter.py            the TM fitter (see the note at its top)
  plots.py             figures
  project.py           settings, folders and summaries shared by the scripts
```

`fitter.py` comes from the LaBr3Wave TM code (github.com/pkuNucExp/LaBr3Wave) with local changes; upstream has no licence, so please do not redistribute it publicly.